Design patterns and backend architecture

Chapter 5: Worked design: submit a commercial payment

Practice designs, not claims about an employer.

5. Worked design: submit a commercial payment

This is a **practice scenario**, not a claim about your employer's implementation. The interviewer asks: "Design the backend for submitting a commercial payment instruction. A customer might retry after a timeout. Downstream processing is asynchronous."

Step 1: Define the contract

Ask: What is the final business outcome? Is the API acknowledging receipt or settlement? How are tenant, amount, currency, beneficiary and authorization validated? What throughput and response-time goal matters? How long should duplicate keys be remembered? Which downstream outcome can be uncertain?

Proposed contract: `POST /payment-instructions` accepts a tenant-scoped idempotency key and a validated request. A successful response means **accepted for processing**, not settled. Return a durable instruction ID and a status resource. A repeated same-key/same-payload request returns the recorded outcome; same key with a different payload is a conflict.

Step 2: Draw the main flow

```
Client
  -> API: authenticate; authorize tenant and account; validate request
  -> Application service: check idempotency and business rules
  -> Local database transaction: instruction + key/result + outbox event
  -> Response: accepted instruction ID and current status

Outbox relay -> durable broker -> payment processor -> provider
                                        |                  |
                                        +-> status update <-+
Status API -> authoritative instruction state
```

The core service depends on a payment store and an external gateway **port**. The REST controller, database repository, outbox relay, and provider client are **adapters**. The domain model protects status transitions. A local transaction protects instruction, deduplication record and outbox record. A separate worker handles the external call.

Step 3: Walk through failures

Failure 1: Client retries after response was lost

Expected behavior: Return the original instruction for the same tenant, key, and normalized payload.

Proof / test: Crash immediately after commit, then retry: ID stays the same; no second charge.

Failure 2: Two identical requests arrive together

Expected behavior: Use a durable uniqueness constraint, not just check-then-insert.

Proof / test: Send parallel requests from separate instances: one instruction and one outbox fact.

Failure 3: Database commit fails

Expected behavior: Roll back instruction, idempotency record, and outbox together.

Proof / test: Force a DB failure: none of the rows remain and acceptance is not returned.

Failure 4: Relay publishes twice

Expected behavior: Make consumer effects durable and idempotent by event/business key.

Proof / test: Deliver the event twice: one applied business effect, duplicate observed.

Failure 5: Provider times out

Expected behavior: Treat the outcome as unknown; query status or reconcile before resubmitting.

Proof / test: Simulate a provider action followed by lost response: no blind second submission.

Failure 6: Downstream backlog grows

Expected behavior: Bound worker concurrency; expose PENDING; alert on oldest-message age.

Proof / test: Burst-load: work stays bounded; queue depth and oldest age are visible.

Failure 7: Permission is revoked

Expected behavior: Recheck authorization for each new operation; do not trust stale role data.

Proof / test: Try cross-tenant access and a revoked role: both are denied.

Step 3A: A transaction boundary in code

```
@Service
final class AcceptInstruction {
    private final InstructionRepository instructions;
    private final OutboxRepository outbox;

    @Transactional
    public Receipt accept(Instruction instruction) {
        // Unique constraint on (tenant_id, idempotency_key).
        Receipt receipt = instructions.insertOnce(instruction);
        outbox.insert(PaymentAccepted.of(receipt.id()));
        return receipt; // This means ACCEPTED, not SETTLED.
    }
}
```

Why this is not the whole solution: The actual duplicate-key conflict must return the previously recorded response only after verifying a matching payload fingerprint. The relay and consumers must be safe for repeated delivery. The remote clearing provider is outside this database transaction. Test the concurrent same-key case and the rollback case with a real database.

Step 4: Explain tradeoffs

Synchronous provider submission could give faster final feedback, but lengthens request latency and couples availability to the provider. Asynchronous submission gives a quick durable acceptance, but customers need a status view and understand pending outcomes. A local database transaction is

enough for local state and the outbox; it cannot atomically include the remote provider. The design needs provider-side idempotency or reconciliation for uncertain outcomes.

Polished 90-second answer: "I would first separate accepting the instruction from final processing. The API authenticates the caller, verifies tenant and account access, and validates the request. I would require a tenant-scoped idempotency key and store the instruction, deduplication result, and outbox event in one local transaction. The response would say 'accepted' with an instruction ID. A worker publishes and processes the event, with duplicate-safe handling and monitored retries. The hardest case is a provider timeout: I would treat that as an uncertain outcome, check the provider using a stable reference or reconcile it, and avoid a blind second charge. I would measure pending age, error rate, duplicate attempts, and final outcomes. This adds asynchronous workflow complexity, but it isolates the API from provider latency and gives a clear recovery path."

Step 5: Tie back to your real experience

Your resumes discuss payment modernization, Java APIs, identity, caching, reliability, and reconciliation. Choose **one** real project and fill in: Which parts of this practice design did you personally build? Which parts were already present? Which failure actually occurred? Which metric can you verify? Do not present this example system as one you implemented unless it matches your work.